

Análise de Desempenho de Algoritmos

Ordenação $O(n \log n)$ e $O(n \log^2 n)$

Estudante: Gustavo Ribeiro Schwert | GSchwert@edu.unisinos.br

Universidade do Vale do Rio dos Sinos (Unisinos)

Análise e Projeto de Algoritmos — 2026/2

Descrição da atividade:

- Implementação manual dos algoritmos Shell Sort, Quick Sort e Merge Sort.
- Proibição do uso de funções ou bibliotecas nativas de ordenação do Python (como sort/sorted).
- Análise da complexidade de tempo
- Estudo de caso e verificação dos comportamentos teóricos em escala.

- Comparação direta com algoritmos quadráticos de complexidade $O(n^2)$
- Avaliação em dois cenários distintos: vetores desordenados e vetores previamente ordenados (reordenação).
- Tamanho de vetor (n) escalonado: variando de 1k (1.000) até 1.000k (1.000.000).
- Demonstração do ponto de inflexão onde os custos de processamento quadrático inviabilizam sistemas.

Ambiente Utilizado:

Especificações de Hardware

- **Processador:** AMD Ryzen 5 5600
- **Memória RAM:** 16 GB RAM DDR4 rodando a 3000 MHz
- **Placa de Vídeo:** NVIDIA GeForce RTX 4060 Externa
- **Armazenamento:** 1 TB SSD NVMe

Especificações de Software

- **Linguagem:** Python 3.13.2
- **Interpretador:** CPython
- **Sist. Operacional:** Windows 11 Pro, Versão 25H2

Projeto de Algoritmo:

1. Entrada de Dados

Vetores com elementos gerados de forma aleatória.

- Valores inteiros gerados na faixa de 0 a 10.500 usando `random.randint()`.
- Uso da lista nativa do Python como estrutura de vetor de inteiros.



2. Execução dos Algoritmos

- Implementação manual de Shell, Quick e Merge Sort.
- Execução individual do teste para cada algoritmo com cada n .
- Testes duplicados: Cenário Desordenado e Cenário Já Ordenado.



3. Escalonamento e Coleta

- Tamanhos testados (n): 1k, 5k, 10k, 50k, 100k, 500k, 1000k.
- Tempo de execução capturado pelo temporizador.

Análise dos dados obtidos para os vetores desordenados

Algoritmo	n = 1k	n = 10k	n = 100k	n = 1000k
Bubble Sort	45,39 ms	4.065,84 ms	387.365,03 ms	Excedeu 600s
Insertion Sort	18,23 ms	1.934,20 ms	174.926,81 ms	Excedeu 600s
Shell Sort	1,38 ms	21,65 ms	394,51 ms	5.926,94 ms
Quick Sort	1,28 ms	13,87 ms	232,98 ms	6.017,81 ms
Merge Sort	1,39 ms	18,70 ms	215,62 ms	2.791,02 ms

Gargalo dos $O(n^2)$

- Bubble Sort leva mais de 6,4 minutos (387.365 ms) em 100k elementos.
- Algoritmos $O(n \log n)$ resolvem 1 milhão de elementos desordenados em um alguns segundos.
- Merge Sort lidera com consistência de 2,79 segundos (2.791 ms) para 1M de elementos.

Análise dos dados obtidos para os vetores já ordenados

Algoritmo	n = 1k	n = 10k	n = 100k	n = 1000k
Bubble Sort	0,06 ms	0,57 ms	7,80 ms	Excedeu 600s
Insertion Sort	0,07 ms	1,11 ms	12,39 ms	Excedeu 600s
Shell Sort	0,74 ms	8,51 ms	170,81 ms	2.843,86 ms
Quick Sort	1,09 ms	14,02 ms	232,98 ms	6.329,98 ms
Merge Sort	1,05 ms	11,76 ms	139,08 ms	1.849,09 ms

Anomalia nos Ordenados

- Bubble e Insertion Sort são velozes em vetores pequenos já ordenados (n=100k abaixo de 13 ms).
- Shell Sort cai de 5.926 ms (desordenado) para 2.843 ms (ordenado) pelo menor número de trocas.
- Merge Sort atinge 1.849 ms no cenário ordenado

Hipóteses sobre a eficácia de cada algoritmo

Shell Sort

Complexidade Teórica: $O(n \log^2 n)$

- Consistentemente o mais lento entre os três eficientes a partir de $n = 10k$.
- Tempo de 5.926 ms para 1M de elementos desordenados.
- Forte otimização na entrada ordenada, caindo para 2.843 ms.

Quick Sort

Complexidade Teórica: $O(n \log n)$

- Excelente velocidade em volumes moderados devido a baixos fatores constantes.
- Alcança 6.017 ms em dados desordenados e 6.329 ms em ordenados.
- Usa escolha aleatória de pivô para estabilizar comportamento prático.

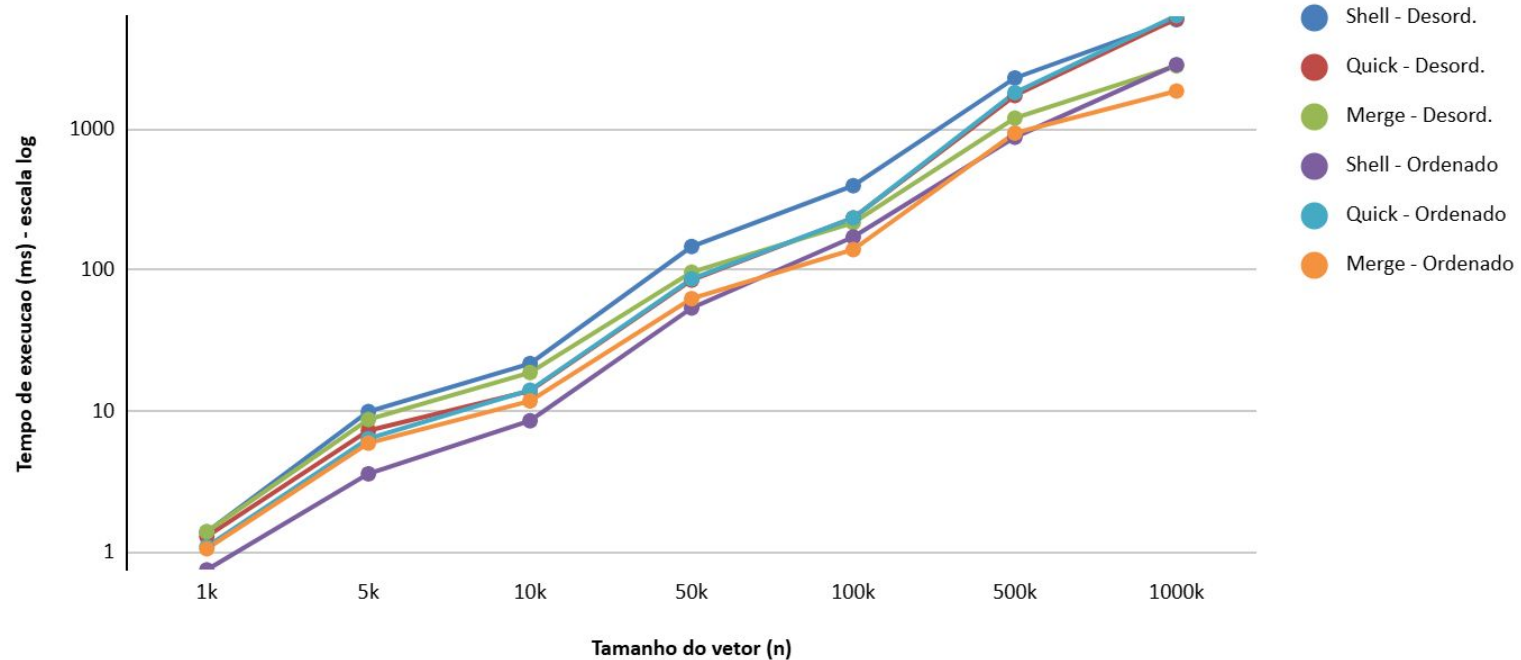
Merge Sort

Complexidade Teórica: $O(n \log n)$

- O algoritmo mais rápido no pior cenário e em volumes extremos de dados.
- Completa 1M desordenados em 2.791 ms e ordenados em apenas 1.849 ms.
- Desempenho garantido teoricamente sem vulnerabilidade a pior caso.

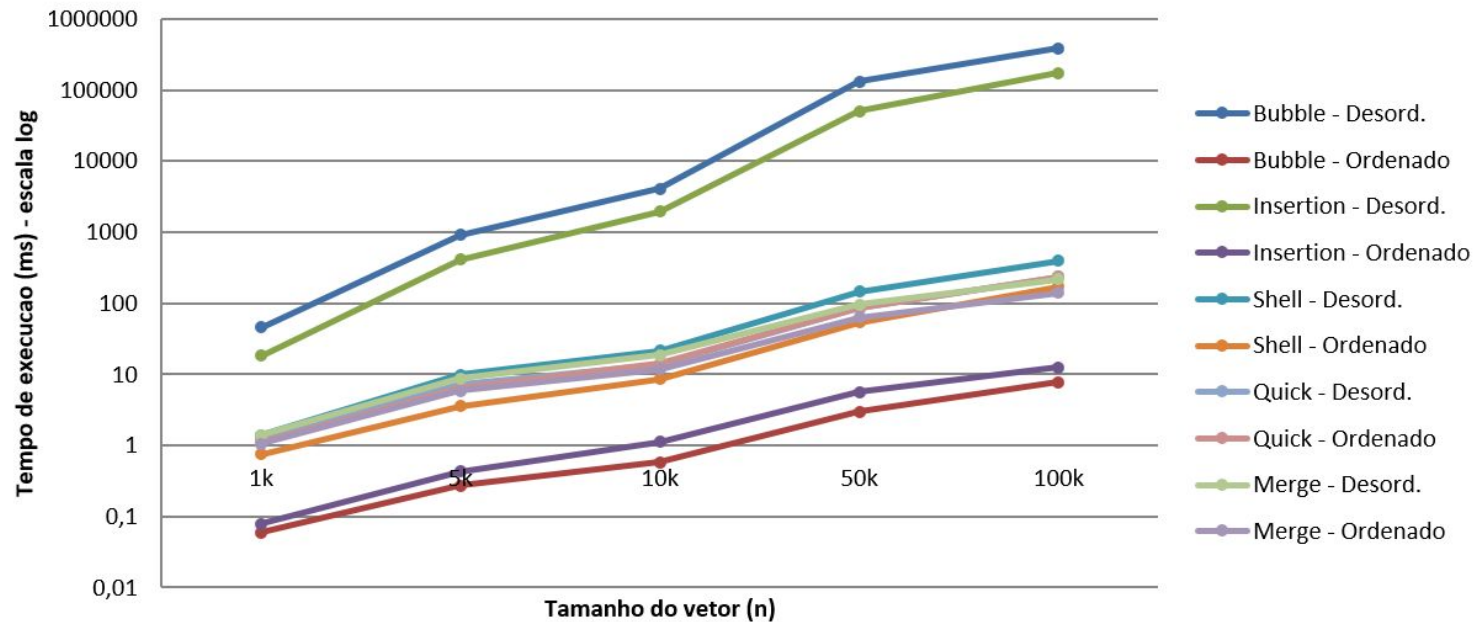
Gráficos dos dados obtidos:

Shell x Quick x Merge Sort - Desordenado x Ja Ordenado



Gráficos dos dados obtidos:

Comparacao de Todos os Algoritmos - Desordenado x Ja Ordenado



Muito Obrigado!

Perguntas?